

CSCE 605 COMPILER DESIGN · MODULE 03 · LECTURE NOTES

Parsing I: Top-Down

Grammars, and the parser you write by hand.

MODULE OUTCOMES

- Write an unambiguous context-free grammar.
- Compute FIRST and FOLLOW sets and test for LL(1).
- Write a recursive-descent parser.
- Handle expression precedence with Pratt parsing.
- Produce error messages and recover to parse the rest.

1 Grammars

```
AMBIGUOUS:  E -> E + E | E * E | ( E ) | num
            "1 + 2 * 3" has TWO parse trees.

STRATIFIED BY PRECEDENCE -- unambiguous:
  E -> E + T | T          (+ lowest, left-associative)
  T -> T * F | F          (* binds tighter)
  F -> ( E ) | num

BUT left recursion loops a top-down parser forever:
  parse_E() calls parse_E() having consumed nothing.

LEFT RECURSION ELIMINATED -- now LL(1):
  E  -> T E'
  E' -> + T E' | epsilon
  ...and the tree now leans RIGHT, so the parser must
  re-associate while building it.
```

Three separate problems are tangled here: the grammar is ambiguous, top-down parsing cannot handle left recursion, and left recursion is exactly how left associativity is expressed. **Fixing the first two damages the third** — the LL(1)-parseable grammar builds a right-leaning tree for a left-associative operator, so the parser has to rebuild the association as it goes. **This tension is precisely what Pratt parsing (§3) sidesteps**, which is why it is worth seeing the problem before the solution.

AMBIGUITY IS A PROPERTY OF THE GRAMMAR, NOT THE LANGUAGE

A *language* is a set of strings. A *grammar* is one particular way of describing that set, and the same language generally has many grammars.

An **ambiguous grammar** assigns some string two or more distinct parse trees — but it is usually possible to find a different grammar for the same language that does not, as the precedence stratification above does.

So **ambiguity is usually a fixable defect of your description**, and the fix is normally to encode the disambiguating rule (precedence, associativity) into the grammar's structure.

Two caveats worth knowing. Some context-free languages are *inherently* ambiguous — no unambiguous grammar exists for them at all. And **determining whether an arbitrary context-free grammar is ambiguous is undecidable**, which is why parser generators report conflicts (a sufficient condition for trouble) rather than proving your grammar unambiguous. That undecidability result is CSCE 627 material, and this is where it has practical consequences.

2 LL(1) and recursive descent

$$\text{FIRST}(\alpha) = \{ t : \alpha \Rightarrow^* t... \} \quad \text{FOLLOW}(A) = \{ t : S \Rightarrow^* ...A t... \}$$

FIRST(α) is the set of tokens that can begin a string derived from α , including ϵ when α can derive the empty string. **FOLLOW**(A) is the set of tokens that can immediately follow A in some derivation, which is needed to decide when to take an ϵ production. **A grammar is LL(1) when, for every pair of alternative productions $A \rightarrow \alpha \mid \beta$, the sets FIRST(α) and FIRST(β) are disjoint** — and additionally, if β can derive ϵ , that FIRST(α) and FOLLOW(A) are disjoint. **The condition says exactly this: one token of lookahead always determines which production to take**, which is precisely what lets a recursive-descent parser decide with a switch on the next token.

```
Node* parse_statement() {
    switch (peek().kind) {
    case TOK_IF:      return parse_if();
    case TOK_WHILE:   return parse_while();
    case TOK_RETURN:  return parse_return();
    case TOK_LBRACE:  return parse_block();
    default:          return parse_expr_statement();
    }
}

Node* parse_if() {
    Span s = expect(TOK_IF).span;
    expect(TOK_LPAREN);
    Node* cond = parse_expr();
    expect(TOK_RPAREN);
    Node* then = parse_statement();
    Node* els  = match(TOK_ELSE) ? parse_statement() : nullptr;
    return make_if(cond, then, els, s.to(prev().span));
}
```

The grammar is the code: one function per nonterminal, one branch per production, one call per symbol. **And the call stack is the parse tree**, which means a stack trace from a recursive-descent parser tells you exactly where in the grammar you are — an underrated advantage over table-driven parsing (Module 04), where the equivalent information is a stack of integers.

3 Pratt parsing

PRECEDENCE CLIMBING HANDLES EXPRESSIONS DIRECTLY

Stratifying a grammar by precedence requires one nonterminal per level. C has roughly fifteen precedence levels, which means fifteen functions each of which does almost nothing except call the next one — and every expression parse walks all fifteen frames even to read a single integer.

Pratt parsing replaces all of them with one function and a table. Each operator is given a binding power; the parser loops while the next operator binds at least as tightly as the current minimum, recursing to collect the right operand.

Associativity falls out of making the binding powers asymmetric. A left-associative operator recurses with a minimum one higher than its own, so an equal-precedence operator to the right stops and is handled by the current loop iteration — producing left nesting. A right-associative operator recurses with the same minimum, so it keeps going right.

And adding a new operator is a table entry, not a new nonterminal. This is what most hand-written compilers actually do for expressions, including Clang and the Go and Rust front ends, while using ordinary recursive descent for statements and declarations.

```
Node* parse_expr(int min_bp) {
    Node* lhs = parse_prefix();          // literal, ident, unary, (expr)
    for (;;) {
        Op op = peek_operator();
        if (!op || left_bp(op) < min_bp) break;
        advance();
        // LEFT-assoc -> recurse with left_bp + 1
        // RIGHT-assoc -> recurse with left_bp
        Node* rhs = parse_expr(right_bp(op));
        lhs = make_binary(op, lhs, rhs);
    }
    return lhs;
}
```

4 Errors and recovery

REPORT MORE THAN ONE ERROR PER RUN

A parser that stops at the first error forces one edit-compile-wait cycle per mistake. On a file with eight typos that is eight round trips, and it is an entirely avoidable experience.

Panic-mode recovery is the standard technique: on encountering an error, report it, then skip tokens until reaching one in the current construct's FOLLOW set — a semicolon, a closing brace, or a statement-introducing keyword.

Then resume normally. The tree contains a hole marked by an explicit error node, which later phases must be prepared to see and skip rather than crash on. **Designing the AST with an error node from the start is much easier than adding one later.**

Suppress cascades. After reporting an error, suppress further error reports until at least one token has been successfully consumed in the normal way. **Without this, one misplaced brace produces forty messages of which only the first is true,** and the user learns to read only the first line — which defeats the recovery entirely.

- **The source location, as a span rather than a point.** Underline the whole offending construct; a caret under one character is rarely enough to see what the compiler objected to.
- **What was expected and what was found, in that order,** phrased in the language's own vocabulary rather than the grammar's. 'Expected a closing parenthesis' beats 'expected RPAREN', which beats 'expected token 47'.
- **The enclosing context, with a second span.** 'Unclosed parenthesis — opened here' pointing at the opening bracket is frequently more useful than the position of the error itself.
- **A suggestion, when one is defensible.** A missing semicolon at the end of the previous line is usually unambiguous and worth stating. **A wrong suggestion is worse than none,** so only offer one when the grammar genuinely determines it.
- **And never the parser's internals.** Token numbers, nonterminal names, and state numbers are bug reports about your compiler, not about the user's program. **Diagnostics are the compiler's entire user interface,** and Rust's reputation for usability rests substantially on having treated them as a first-class deliverable rather than an afterthought.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Nystrom — Crafting Interpreters, 'Parsing Expressions' and 'Compiling Expressions' (free)	Recursive descent and then Pratt parsing, both implemented in full.

Resource	What it covers for this module
Matklad — Simple but Powerful Pratt Parsing (free)	The clearest explanation of §3 available , including the binding-power asymmetry that everyone gets wrong first.
Stanford CS143 — top-down parsing lectures (free)	FIRST, FOLLOW, and the LL(1) condition of §2, derived.
Rust — the rustc diagnostics guide (free)	§4 taken seriously by people who made it a priority . Worth reading for the standards it sets.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write an ambiguous expression grammar and exhibit two parse trees for one string.
- 2 Stratify it by precedence and show the ambiguity is gone.
- 3 **Compute FIRST and FOLLOW** for your project grammar by hand, then write code to compute them and compare.
- 4 Test your grammar for LL(1) and fix any violations.
- 5 Write a recursive-descent parser for statements and declarations.
- 6 **Implement Pratt parsing** for expressions with at least eight precedence levels.
- 7 **Demonstrate associativity** by parsing $a - b - c$ and $a = b = c$ and printing both trees.
- 8 Add a new operator by adding only a table row, and confirm nothing else changed.
- 9 Implement panic-mode recovery and cascade suppression.
- 10 **Build a diagnostics gallery** of twenty ill-formed programs with the message each produces, and improve the five worst.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is ambiguity a property of the grammar, and what is undecidable about it?
- 2 Why does left recursion break top-down parsing, and what does eliminating it cost?
- 3 Define FIRST and FOLLOW and state the LL(1) condition.
- 4 Why is the call stack of a recursive-descent parser useful?
- 5 What problem does Pratt parsing solve, and how does it encode associativity?
- 6 Describe panic-mode recovery and why cascades must be suppressed.

7 Name five properties of a good diagnostic.